Audion Office OCR AI

Version 1.8.0 · 2026-08-24 · 971.8 MB

- Direct download — unmetered, no rate limits
- Project page — every version and how to install

`SHA-256: aecbb47c031856ea5faac64c09356664060d8418d0d8e03e4d623f8da4d50645`

Русская версия

Portable Windows OCR and office-conversion platform. It stores scans in a provider-neutral `DocumentModel` and locally builds editable and archival formats without rerunning paid OCR.

Core loop:

```
source files -> OCR Brick -> DocumentModel -> review/corrections -> independent exporters
```

Markdown remains a useful audit and LLM format, but it is not the center of the OCR system.

Provider-neutral DocumentModel

Each `<name>.document` package preserves the exact source, page images before and after preprocessing, hashes, text and coordinates from all OCR candidates, reading order, tables and merges, confidence, Mistral/Tesseract verification, raw provider results, and manual-correction slots. Completeness means no loss of source data, not a promise of 100% OCR accuracy.

The primary OCR GUI can independently build eight results in one run: `DOCX`, `XLSX`, `Searchable PDF`, `ODT`, `Markdown`, `OCR JSON`, `HTML`, and `Verification`. DOCX is the only working default and there are no format presets. Full archive ZIP remains a backend compatibility capability but is not shown among the primary chips. Re-export is local and does not require LibreOffice or another OCR request. DOCX page size is capped at A3.

Mistral OCR 4 exposes an independent second-pass selector in the GUI: `None`, `Tesseract`, `Yandex`, or `Yandex + Tesseract`. For ruled tables the backend recovers a physical grid from long raster lines and verifies its column count against the numbered `1…N` row from coordinate-aware OCR words. Mistral, Yandex and Tesseract are then compared inside exact physical cells. Merges are recovered only in the header; numeric disagreements remain review items.

Canonical Workbench labels

The address rows are `Source` and `Target`. The action labels are identical across Audion NiceGUI projects: `Source`, `Add file...`, `Target`, `Reset`, `Delete`, and `List`. `Source` can be one file or a folder; external input is mirrored into managed `input`, and results are synchronized to the selected `Target`.

CLI/FZF and GUI use the same manifest/service layer through `system_core/cli_operation.py`; OCR Brick, Workbench resolvers, and office builders do not maintain separate backend copies.

Current Status

- `launcher_project.cmd` remains the stable English CLI/FZF backend.
- `launcher_project_ru.cmd` provides the matching Russian CLI/FZF backend.
- `launcher_gui.cmd` provides the portable NiceGUI operator shell.
- Local extraction lives in `system_core/extract_to_md.py`; the new OCR Brick lives in `system_core/ocr_brick`.
- Markdown builders exist for `DOCX`, Word-based `PDF`, `PPTX`, and `XLSX`.
- A separate DEV Markdown PDF engine renders paired dark/light PDFs through Playwright Chromium.
- OCR Brick combines scan preprocessing, local OCR, API OCR, one-page quality tests and legal/financial requisites checks.
- The main product goal is a complete table-first OCR cycle for PDF/image scans, especially coordinate-heavy, numeric and legal tables.

Capabilities

- `DOCX`, `XLSX`, `PPTX`, `PDF`, `TXT`, `CSV`, `HTML` -> Markdown.
- `PDF`, `JPG`, `PNG`, `TIFF`, `WEBP` -> `DocumentModel` through local OCR or API OCR, then directly into selected formats.
- Local OCR: Tesseract is recommended; Surya remains optional for slow free runs.
- API OCR: Yandex, Mistral OCR 4, xAI, Gemini and OpenAI through the common OCR Brick contract.
- Gemini supports `Standard`, `Flex` and separate `Gemini Batch OCR` for large non-urgent runs.
- One-page OCR quality tests compare raw/clean and local/API engines before large jobs.
- Requisites Check verifies exact contract numbers, purchase IDs, dates, sums and similar legal/financial strings.
- Markdown table extraction into Excel.
- Coordinate/table checks and exports to Excel.
- Markdown builds into office DOCX, dense LLM-DOCX, Word-based PDF, PPTX, and XLSX.
- DEV Markdown PDF builds paired `dark` and `light-sand` PDFs from selected Markdown documents through Chromium. It can discover project docs, rebuild existing PDF pairs, process only outdated PDFs, and write output to a per-folder `PDF` directory beside Markdown, directly beside Markdown, shared `docs/PDF`, or the current Workbench Target tree.
- GUI `Source` and `Destination` fields with recursive mirroring:
  - external source -> managed `input`
  - `output` -> external destination
- Machine-readable reports under `report`, separate from user-facing `output`.

Quick Start

Build or refresh the portable environment:

```
builder_main.cmd
```

or directly:

```
install\Build_Portable_Env_Build.cmd
```

Build/install scripts keep the portable GUI template structure, with optional component entries for Playwright Chromium used by DEV Markdown PDF, portable Tesseract, Real-ESRGAN ncnn-vulkan and optional Surya acceleration.

Run the GUI:

```
launcher_gui.cmd
```

Run the CLI/FZF launcher:

```
launcher_project.cmd
```

Run the Russian CLI/FZF launcher:

```
launcher_project_ru.cmd
```

The first CLI/FZF entries mirror the current GUI workflows: office text without OCR, Tesseract/Surya, Yandex/Mistral/xAI/Gemini/OpenAI, local Workbench, OpenAI/Gemini resolvers, coordinates, and verified Mistral/Yandex/Tesseract fusion. Every route goes through `system_core/cli_operation.py` and the same `office_service.py`.

Typical workflow:

1. Put documents into `input` or choose an external `Source` in the GUI.
2. For text-based office files, use `Office Text Without OCR`.
3. For scans, run `OCR Quality Test` on one page first.
4. Choose `Local OCR` or `Paid API OCR`, cleanup profile and model.
5. For legal documents, also run `Requisites Check`.
6. In `Deliverables`, keep DOCX or select XLSX, Searchable PDF, ODT and audit formats together.
7. Review generated DOCX/XLSX in `output` and machine reports in `report`.
8. Use `COPY output TO input` and legacy Markdown compilers only for archived compatibility workflows.
9. Use `DEV Markdown PDF` when users need readable dark/light PDF copies of Markdown documentation.

GUI

The GUI is a shell over the existing core, not a separate OCR implementation.

The GUI currently provides:

- `Source` and `Destination` path fields with picker buttons
- recursive input/output mirroring
- live terminal log
- root windows for `Office Text Without OCR`, `Local OCR`, `Paid API OCR`, `OCR Quality Test`, `Requisites Check`, `Tables`, and `Project Tools`
- model and local key-file dropdowns per API provider
- online model-list refresh with cache for OpenAI, Gemini, Yandex and xAI
- pinned model dropdowns for curated model sets
- OCR Brick engines: Tesseract, Surya, Yandex, Mistral OCR 4, xAI, Gemini and OpenAI
- one compact paid-engine row with a single `Engine` heading followed by Yandex, xAI, Mistral OCR 4, Gemini and ChatGPT; very narrow windows scroll this row horizontally instead of producing arbitrary wrapped rows
- one shared `Deliverables` block directly below the engine in local and paid OCR: a shaded 4-by-2 grid of rounded rectangular checkbox chips; the office row is DOCX/XLSX/Searchable PDF/ODT and the audit/developer row is Markdown/OCR JSON/HTML/Verification; there are no presets, DOCX is the working default, and the internal DocumentModel is always preserved
- duplicate `Output formats` and `Scan cleanup` field labels are hidden while section headings and shaded explanatory hint blocks remain
- `Text + boxes` and `Layout` divide the complete OCR contract row into two equal columns
- a confirmed `Clear DocumentModel artifacts` maintenance action removes `*.document`, `*.document.json`, and `*.verification.json` while preserving DOCX/XLSX/Markdown/PDF, images, sources and full archive ZIP
- legacy Markdown compilers are removed from the main screen and remain under `Project tools -> Rebuild legacy Markdown` only for archived compatibility workflows
- Mistral OCR 4 expands table attachments into self-contained Markdown and safely repairs confirmed CP1251/UTF-8 mojibake; dense composite pages still require `OCR Quality Test` or a local Tesseract sidecar
- OCR Brick outputs can be selected together as checkbox chips (`OCR JSON`, `Markdown`, `Verification`); Mistral + 2-pass Tesseract writes separate verification JSON/Markdown with `pass/review`, numeric agreement and mismatch lists

- Gemini `Standard/Flex` plus `Gemini Batch OCR` / `Забрать Gemini Batch`
- 2-pass Tesseract as a local hint/sidecar for paid OCR
- a flat `Tables` window with three expanded blocks for coordinate rows, Markdown table inspection, and XLSX build
- DOCX layout controls: portrait/landscape orientation, margins in millimeters, system font, body font size in pt
- DEV Markdown PDF controls: what to process, where to search, document presets, custom filename filters, found Markdown selection, output mode, plus advanced page margins, page top/bottom reserve, and line-height
- quick access to `logs`, `report`, and `config`

Parameterized commands open a final screen with `Back` on the left and `Run` on the right. Fast commands without parameters can run immediately.

Service JSON/XLSX/preview artifacts are written to `report`. User-facing deliverables should stay in `output`.

OCR Brick

Main OCR paths:

- local extraction for text-based office formats
- local Tesseract OCR with preprocessing and coordinates
- optional Surya OCR when API cost matters more than elapsed time
- Yandex Vision OCR with `page`, `table`, `markdown`, `handwritten` modes
- xAI vision OCR with curated/pinned models: `4.20 fast` for literal OCR and `4.20 quality` reasoning for difficult tables and fragmented scans
- regular non-streaming `chat/completions`, up to four attempts with backoff for transient HTTP/network failures, strict UTF-8 validation and a resumable per-page cache
- controlled Russian OCR review: off by default; when enabled, the reasoning model reviews only suspicious pages, excessive rewrites are rejected and original text is retained in result metadata
- Gemini/OpenAI vision OCR for quality comparisons and difficult pages
- one-page raw/clean benchmark before large runs
- numeric/legal requisites checker for exact strings

Preprocessing is controlled by the `Scan cleanup` profile: `Auto`, `Raw`, `Heavy scan`, `Numbers`, `Manual`. Detailed manual controls live in Advanced.

Gemini modes are documented in `docs/GEMINI_BATCH_FLEX.md`: `Standard` for interactive OCR, `Flex` for cheaper but less predictable synchronous calls, and `Batch` for large non-urgent jobs.

What To Install For Surya/Acceleration

The detailed table lives in `install/README_INSTALL.md`.

Quick choice:

- RTX 50xx: `[11] REAL-ESRGAN VULKAN`, `[13] SURYA CPU OCR`, `[15] LLAMA.CPP CUDA 13.3`; `[17] SURYA PYTORCH CUDA 50XX` is only a heavy benchmark.
- RTX 40xx: `[11]`, `[13]`, `[14] LLAMA.CPP CUDA 12.4`; `[16] SURYA PYTORCH CUDA 40XX` is only a benchmark.
- AMD / Intel Arc / Vulkan GPU: `[11]`, `[13]`, `[12] LLAMA.CPP VULKAN`.
- CPU-only: `[13]` only when Surya is required; Tesseract or API OCR is often more practical.

`REAL-ESRGAN VULKAN` improves images before OCR, but does not recognize text. `SURYA CPU OCR` installs Surya itself. `LLAMA.CPP` is the lightweight Surya acceleration backend. `SURYA PYTORCH CUDA` is a heavy optional benchmark, not the default path.

Optional Portable Tesseract

`pytesseract` installs only the Python bridge. Native Tesseract is not part of the core bundle. The project starts without it; local Tesseract OCR and 2-pass checks are unavailable until Tesseract is installed.

When local OCR is needed, install Tesseract as an optional portable component. The project looks inside the portable runtime first:

```
runtime\tesseract\tesseract.exe
runtime\tesseract\tessdata\eng.traineddata
runtime\tesseract\tessdata\rus.traineddata
runtime\tesseract\tessdata\deu.traineddata
runtime\tesseract\tessdata\osd.traineddata
```

Install or refresh the optional component from `builder_main.cmd` -> `Install portable Tesseract`, or run:

```
install\Install-Portable-Tesseract.cmd
```

A fresh core rebuild recreates `runtime\`; run the optional component installer afterwards if you want local Tesseract OCR.

The installer owns only the project-local `runtime\tesseract` folder: every run assembles a fresh temporary payload, downloads the selected language data, removes the previous project copy if it exists, and moves the fresh copy into place.

No system Tesseract installation is required. If no system source exists, the script resolves the latest stable x64 UB Mannheim build and runs the NSIS installer silently into project staging, then creates the portable project copy. If a Windows install such as `C:\Program Files\Tesseract-OCR` already exists, the default path copies from it as a read-only source to avoid NSIS maintenance UI. Do not manually point the UB Mannheim installer UI at the project folder; the project target is fixed as `runtime\tesseract`.

For a non-standard local source, set `AUDION_TESSERACT_SOURCE_DIR` before running the installer.

Lookup order:

1. `runtime\tesseract\tesseract.exe`
2. `AUDION_TESSERACT_EXE`
3. system `PATH`

Verify:

```
runtime\tesseract\tesseract.exe --version
runtime\tesseract\tesseract.exe --list-langs
```

For Russian+English documents, use:

```
rus+eng
```

If Tesseract is unavailable, local Tesseract OCR and 2-pass modes should report the limitation explicitly instead of failing silently.

AI Settings

OCR is configured through:

- `config/llm_settings.yaml`
- `config/ocr_prompt_vision_ocr.md`
- `config/gui_model_pins.json`
- `system_core/ocr_brick/preprocess.profiles.json`

Default key files:

- OpenAI: `config/api_key_openai.txt`
- Gemini: `config/api_key_gemini.txt`
- Yandex: `config/api_key_yandex_studio.txt`
- xAI: `config/api_key_xai.txt`
- Mistral: `config/api_key_mistral.txt`

Additional key files may be stored in:

- provider-specific extra key files where enabled by the GUI

The GUI shows only the key-file path/status, never the key value.

Document Builders

DOCX build supports:

- portrait and landscape orientation
- page margins in millimeters
- body font from the system font list
- body font size in pt
- regular office DOCX
- dense LLM-DOCX

PDF has two separate paths:

- Word-based PDF through `compile_md_to_pdf.py` for office-style output.
- DEV Markdown PDF through `dev_markdown_pdf_engine.py` for paired dark/light documentation PDFs via Chromium.

In the GUI, DEV Markdown PDF is a managed documentation pipeline:

- `What to process`: Markdown from `input/output`, the GUI `Source` file/folder, docs under a path, existing PDF pairs, or outdated PDFs only.
- `Where to search`: relative to the project root or absolute; blank means the current project root.
- `Document presets`: `README`, `USER`, `GUIDE`, `GUI`, `GUARD`, `ARCHITECTURE`, `CHANGELOG`, `INSTALL`, `PORTABLE`, `USAGE`, `CONFIG`, `TROUBLE`, `SECURITY`, with additional opt-in presets such as `FAQ`, `API`, `ROADMAP`, `MIGRATION`, `RELEASE`, `LICENSE`, and `AGENTS`.
- `Custom filename filters`: comma-separated path/name tokens such as `API, ROADMAP, INSTALL`.
- `Found Markdown`: scan, select all, only without PDF, only outdated, only CHANGELOG, or clear.
- `Output mode`: `docs/PDF` by default; alternatives are `PDF next to MD`, directly beside Markdown, or mirrored into the current Workbench Target.
- After export, the terminal prints a short report: `MD`, `PDF`, `Pages`, `Errors`, `Stale after export`.

XLSX can be built directly from the saved DocumentModel without another OCR request. For ruled scans, the backend conservatively recovers a physical grid from raster lines, validates the column count against the coordinate-aware `1…N` row, assigns provider text to verified cells, and leaves ambiguous or numeric disagreements for review. The `Tables` root window also exposes three expanded service blocks for coordinate rows, Markdown table inspection, and legacy XLSX build.

Project Structure

```
Audion Office OCR AI/
├── config/                # API keys, OCR prompts, LLM/OCR settings, GUI settings
├── data/                  # model-list cache and local cache files
├── input/                 # source staging area
├── output/                # user-facing results
├── report/                # machine reports and service artifacts
├── logs/                  # runtime logs
├── install/               # build, install, release scripts
├── system_core/           # Python OCR core, OCR Brick, GUI services
├── runtime/               # portable Python and optional component runtime,
generated locally
├── wheelhouse/            # offline wheels, generated locally
├── release/               # release archives
├── launcher_gui.cmd       # GUI launcher
├── launcher_project.cmd   # EN CLI/FZF launcher
├── launcher_project_ru.cmd # RU CLI/FZF launcher
├── launcher_tools.cmd     # tools, build, diagnostics
└── builder_main.cmd       # portable environment builder
```

Security

Local extraction, builds and local OCR process files on the machine. Local Tesseract uses optional portable Tesseract from `runtime\tesseract` when installed. DEV Markdown PDF rendering is local through portable Chromium. API OCR sends rendered pages/images and, when 2-pass is enabled, local OCR hints to the selected provider. Gemini Batch writes local JSONL/manifest files first; submission to Google happens only when the submit toggle is enabled.

Use API OCR only when your policy allows sending document images to the selected provider.

Do not commit:

- API keys
- `runtime/`
- `wheelhouse/`
- `output/`
- `report/`
- `logs/`

- `release/`
- `workspace/`
- `data/`

Roadmap

- Add deeper OCR Brick table segmentation and table-crop grouping.
- Expand quality/requisites reports into a guided review/edit workflow.
- Add stronger coordinate-table validation before XLSX export.
- Keep CLI/FZF as the stable backend.
- Keep EN/RU launchers in parity.
- Keep the GUI as a thin portable shell over auditable commands.

License

See the `licenses/` directory in the release package for third-party license notices.

Result Acceptance

Always test one representative page before a batch. Review OCR text, tables, requisites, page order, language, and provider warnings in the generated reports. Keep the source scan until the DOCX/XLSX and any selected Markdown export have been accepted.

The GUI manifest is the structured source for OCR workflows, fields, defaults, conditional controls, tooltips, and backend actions. Documentation translates those definitions into engine choice, privacy boundaries, document expectations, and acceptance checks.